

NiceGUI 中 app.router 深度解析

在 NiceGUI 框架中，`app.router` 是基于底层 FastAPI/Starlette 路由系统实现的**请求路由核心组件**，负责将客户端的 HTTP 请求映射到对应的处理函数（如页面函数、API 接口），是实现 URL 与业务逻辑解耦的关键。`app.router` 不仅承载了 NiceGUI 页面路由（`ui.page`）的注册与匹配，还支持自定义 API 路由、动态路由、路由参数解析等高级功能，是连接客户端请求与后端逻辑的“交通枢纽”。本文将从**核心原理**、**路由注册方式**、**页面路由与 API 路由**、**高级路由特性**、**路由匹配规则**、**常见问题**六个维度，全方位解析 NiceGUI 的 `app.router` 机制。

一、app.router 的核心原理

NiceGUI 基于 FastAPI 构建，而 FastAPI 又继承了 Starlette 的路由系统，因此 `app.router` 的底层本质是**Starlette 的 Router 实例**，其核心工作原理可总结为**路由注册与请求匹配**两大阶段：

1.1 路由注册阶段

- 框架自动注册**：NiceGUI 启动时，会自动将通过 `ui.page` 装饰器定义的页面函数注册到 `app.router` 中，生成对应的路由规则（如路径、请求方法、处理函数的映射）；
- 手动注册**：开发者可通过 `app.router` 提供的 API 手动注册自定义路由（如 API 接口、静态资源路由），补充框架自动注册的路由规则；
- 路由规则存储**：所有路由规则被存储在 `app.router.routes` 列表中，每个规则包含**路径模板**、**请求方法**、**处理函数**、**名称**等核心信息。

1.2 请求匹配阶段

当客户端发送 HTTP 请求到 NiceGUI 应用时，`app.router` 会执行以下匹配逻辑：

- 请求解析**：解析请求的**路径**（如 `/dashboard`）和**方法**（如 GET、POST）；
- 路由匹配**：遍历 `app.router.routes` 中的路由规则，按照**精确匹配优先**、**模糊匹配次之**的原则，找到与请求路径和方法匹配的路由规则；
- 参数解析**：若路由规则包含路径参数（如 `/user/{user_id}`），则从请求路径中提取参数并传递给处理函数；
- 执行处理函数**：调用匹配到的处理函数（页面函数或 API 函数），生成响应数据并返回给客户端。

简单来说，`app.router` 的核心作用是**建立 URL 路径与后端处理函数的映射关系**，并在请求到达时完成路径匹配与逻辑分发。

二、app.router 的核心属性与基础 API

`app.router` 作为 Starlette `Router` 实例的直接引用，提供了一系列属性和 API 用于管理路由规则，以下是开发中最常用的核心内容：

2.1 核心属性

属性	类型	作用
<code>app.router.routes</code>	列表	存储所有已注册的路由规则，每个元素为 Starlette 的 <code>Route / Mount / WebSocketRoute</code> 实例

属性	类型	作用
<code>app.router.url_path_for</code>	方法	根据路由名称生成对应的 URL 路径，支持传递路径参数
<code>app.router.default</code>	可选 [处理函数]	未匹配到任何路由时的默认处理函数（404 处理）

2.2 基础路由注册 API

`app.router` 提供了 `add_route`、`add_api_route`、`mount` 等方法用于手动注册路由，替代 `ui.page` 实现更灵活的路由管理：

- 1. `add_route(path, endpoint, methods=["GET"], name=None)`：注册基础 HTTP 路由，支持自定义请求方法；
- 2. `add_api_route(path, endpoint, methods=["GET"], response_model=None)`：FastAPI 专属的 API 路由注册，支持数据模型验证、自动生成接口文档；
- 3. `mount(path, app, name=None)`：挂载子应用（如静态资源服务、其他 FastAPI 应用），实现路由嵌套。

三、NiceGUI 路由的核心注册方式

NiceGUI 提供了 ** 框架封装的声明式注册（`ui.page`）和原生手动注册（`app.router`） ** 两种路由注册方式，分别适用于页面开发和自定义 API 开发的场景，二者可无缝协同。

3.1 声明式注册： `ui.page` 装饰器（页面路由）

`ui.page` 是 NiceGUI 为前端页面开发封装的路由装饰器，底层会调用 `app.router.add_route` 完成路由注册，是最常用的页面路由方式。

3.1.1 基础页面路由注册

```
from nicegui import ui, app

# 注册首页路由：路径 /，名称 index
@ui.page('/', name='index')
def index_page():
    ui.label('NiceGUI 首页').classes('text-3xl font-bold')

# 注册关于页路由：路径 /about，名称 about
@ui.page('/about', name='about')
def about_page():
    ui.label('关于我们').classes('text-2xl')

if __name__ in {'__main__'}:
    # 启动应用时，ui.page 注册的路由已自动添加到 app.router
    ui.run()
```

3.1.2 带路径参数的页面路由

支持在路径中定义动态参数，参数会自动传递给页面处理函数，与 FastAPI 的路径参数语法完全一致：

```
# 带单个路径参数的路由: /user/{user_id}
@ui.page('/user/{user_id}')
def user_page(user_id: str):
    ui.label(f'用户 ID: {user_id}').classes('text-2xl')

# 带类型注解的路径参数: 限制 user_id 为整数, article_id 为字符串
@ui.page('/user/{user_id:int}/article/{article_id:str}')
def article_page(user_id: int, article_id: str):
    ui.label(f'用户 {user_id} 的文章 {article_id}').classes('text-2xl')
```

3.1.3 路由名称与 URL 生成

通过 `name` 参数为路由命名后，可通过 `app.router.url_path_for` 生成对应的 URL 路径，避免硬编码路径：

```
@ui.page('/', name='index')
def index():
    # 根据路由名称生成 about 页的 URL
    about_url = app.router.url_path_for('about')
    # 生成带参数的 user 页 URL
    user_url = app.router.url_path_for('user', user_id=123)

    ui.link('前往关于页', about_url).classes('mt-2')
    ui.link('前往用户123的页面', user_url).classes('mt-2')

@ui.page('/about', name='about')
def about():
    pass

@ui.page('/user/{user_id}', name='user')
def user(user_id: str):
    pass
```

3.2 手动注册： `app.router` 原生 API（API 路由 / 自定义路由）

对于非页面的后端 API 接口，可通过 `app.router` 的原生 API 手动注册路由，实现前后端分离的接口开发，同时支持 FastAPI 的所有 API 特性（如请求体验证、响应模型）。

3.2.1 注册基础 HTTP 接口

```
from nicegui import ui, app
from starlette.responses import JSONResponse

# 定义 API 处理函数
async def get_user(request):
    # 从路径参数中提取 user_id
    user_id = request.path_params.get('user_id')
    return JSONResponse({'user_id': user_id, 'name': '张三', 'age': 20})
```

```

# 手动注册 GET 接口: /api/user/{user_id}
app.router.add_route(
    path='/api/user/{user_id}',
    endpoint=get_user,
    methods=['GET'],
    name='api_user'
)

@ui.page('/')
def index():
    # 调用自定义 API
    ui.button('获取用户信息', on_click=lambda:
ui.run_javascript('fetch("/api/user/123").then(res=>res.json()).then(console.log)'))

if __name__ in {'__main__'}:
    ui.run()

```

3.2.2 注册 FastAPI 风格的 API 接口

利用 `app.router.add_api_route` 可注册支持数据模型验证的 API 接口，自动生成 OpenAPI 文档（访问 `/docs` 查看）：

```

from nicegui import ui, app
from pydantic import BaseModel

# 定义请求体模型
class UserCreate(BaseModel):
    name: str
    age: int
    email: str | None = None

# 定义响应体模型
class UserResponse(BaseModel):
    id: int
    name: str
    age: int
    email: str | None = None

# 定义 API 处理函数
def create_user(user: UserCreate):
    # 模拟创建用户，生成 ID
    return UserResponse(id=1, name=user.name, age=user.age, email=user.email)

# 手动注册 POST API，支持请求体验证和响应模型
app.router.add_api_route(
    path='/api/users',
    endpoint=create_user,
    methods=['POST'],
    response_model=UserResponse,
    name='create_user'
)

```

```
if __name__ in {'__main__'}:
    ui.run() # 启动后访问 http://localhost:8080/docs 查看接口文档
```

3.2.3 挂载子应用 / 静态资源路由

通过 `app.router.mount` 可挂载静态资源服务或其他 FastAPI 子应用，实现路由的嵌套管理，补充 NiceGUI 内置的静态资源托管能力：

```
from nicegui import ui, app
from fastapi.staticfiles import StaticFiles

# 挂载静态资源: /media 路径映射到本地 ./media 文件夹
app.router.mount(
    path='/media',
    app=StaticFiles(directory='media'),
    name='media'
)

@ui.page('/')
def index():
    # 访问挂载的静态资源
    ui.image('/media/photo.jpg').classes('w-64 h-64')

if __name__ in {'__main__'}:
    ui.run()
```

四、app.router 的高级路由特性

`app.router` 继承了 Starlette/FastAPI 的所有高级路由特性，包括路由前缀、路由依赖、404/500 异常处理、WebSocket 路由等，可满足复杂应用的路由需求。

4.1 路由前缀与路由分组

通过创建独立的 `Router` 实例并挂载到 `app.router`，可实现路由前缀和分组，便于管理模块化的 API 接口：

```
from nicegui import ui, app
from fastapi import APIRouter
from starlette.responses import JSONResponse

# 创建带前缀的 API 路由器
api_router = APIRouter(prefix='/api/v1')

# 为子路由器注册接口
@api_router.get('/users')
def get_users():
    return JSONResponse([{'id': 1, 'name': '张三'}, {'id': 2, 'name': '李四'}])

@api_router.get('/articles')
def get_articles():
    return JSONResponse([{'id': 1, 'title': 'NiceGUI 教程'}])
```

```
# 将子路由器挂载到 app.router
app.router.include_router(api_router)

@ui.page('/')
def index():
    ui.label('API 分组示例').classes('text-3xl')
    # 访问 /api/v1/users 和 /api/v1/articles

if __name__ in {'__main__'}:
    ui.run()
```

4.2 自定义 404/500 异常处理路由

通过 `app.router.add_exception_handler` 可注册自定义的异常处理函数，替换框架默认的 404/500 错误页面，提升用户体验：

```
from nicegui import ui, app
from starlette.responses import HTMLResponse
from starlette.exceptions import HTTPException
from fastapi import Request

# 自定义 404 异常处理函数
async def not_found_handler(request: Request, exc: HTTPException):
    return HTMLResponse(
        content='<h1>404 - 页面未找到</h1><p>你访问的路径不存在，请检查 URL! </p>',
        status_code=404
    )

# 自定义 500 异常处理函数
async def server_error_handler(request: Request, exc: HTTPException):
    return HTMLResponse(
        content='<h1>500 - 服务器内部错误</h1><p>请稍后再试! </p>',
        status_code=500
    )

# 注册异常处理函数
app.router.add_exception_handler(404, not_found_handler)
app.router.add_exception_handler(500, server_error_handler)

@ui.page('/')
def index():
    ui.button('访问不存在的页面', on_click=lambda: ui.navigate.to('/nonexistent'))

if __name__ in {'__main__'}:
    ui.run()
```

4.3 WebSocket 路由注册

NiceGUI 底层依赖 WebSocket 实现前后端实时通信，通过 `app.router.add_websocket_route` 可手动注册自定义 WebSocket 路由，实现更灵活的实时交互：

```
from nicegui import ui, app
from starlette.websockets import WebSocket

# 自定义 WebSocket 处理函数
async def echo_websocket(websocket: WebSocket):
    await websocket.accept()
    while True:
        # 接收客户端消息
        data = await websocket.receive_text()
        # 回声发送
        await websocket.send_text(f'服务器收到: {data}')

# 注册 WebSocket 路由: /ws/echo
app.router.add_websocket_route('/ws/echo', echo_websocket)

@ui.page('/')
def index():
    # 前端连接 WebSocket 并测试
    ui.run_javascript('''
        const ws = new WebSocket(`ws://${window.location.host}/ws/echo`);
        ws.onopen = () => ws.send('Hello NiceGUI!');
        ws.onmessage = (e) => console.log('WebSocket 消息: ', e.data);
    ''')
    ui.label('WebSocket 回声测试，请查看浏览器控制台').classes('text-2xl')

if __name__ in {'__main__'}:
    ui.run()
```

4.4 路由依赖注入

利用 FastAPI 的依赖注入特性，可为路由添加通用的前置逻辑（如权限验证、参数解析），实现路由的复用与解耦：

```
from nicegui import ui, app
from fastapi import Depends, HTTPException
from starlette.responses import JSONResponse

# 定义依赖函数：验证令牌
def verify_token(token: str = 'default_token'):
    if token != 'valid_token':
        raise HTTPException(status_code=401, detail='令牌无效')
    return token

# 注册带依赖的 API 路由
app.router.add_api_route(
    path='/api/protected',
    endpoint=lambda token: JSONResponse({'message': '受保护的接口', 'token': token}),
    methods=['GET'],
```

```

dependencies=[Depends(verify_token)]
)

@ui.page('/')
def index():
    # 访问受保护的接口（带有效令牌）
    ui.button('访问受保护接口', on_click=lambda: ui.run_javascript('fetch("/api/protected?token=valid_token").then(res=>res.json()).then(console.log)'))
    # 访问受保护的接口（带无效令牌）
    ui.button('访问无效令牌接口', on_click=lambda: ui.run_javascript('fetch("/api/protected?token=invalid").then(res=>res.json()).then(console.log)'))

if __name__ in {'__main__'}:
    ui.run()

```

五、app.router 的路由匹配规则

`app.router` 的路由匹配遵循 Starlette/FastAPI 的核心规则，理解这些规则可避免路由冲突、匹配失效等问题，以下是关键匹配规则：

5.1 精确匹配优先于模糊匹配

路由系统会优先匹配**精确路径**，再匹配**带参数的模糊路径**，最后匹配**通配符路径**。示例：

```

# 精确路径: /user
@ui.page('/user')
def user_index():
    ui.label('用户首页')

# 模糊路径: /user/{user_id}
@ui.page('/user/{user_id}')
def user_detail(user_id: str):
    ui.label(f'用户 {user_id}')

# 通配符路径: /user/{path:path}
@ui.page('/user/{path:path}')
def user_wildcard(path: str):
    ui.label(f'用户通配符: {path}')

```

- 访问 `/user` → 匹配 `user_index`（精确匹配）；
- 访问 `/user/123` → 匹配 `user_detail`（参数模糊匹配）；
- 访问 `/user/123/article` → 匹配 `user_wildcard`（通配符匹配）。

5.2 路径参数的类型约束

路径参数支持类型注解（`int/str/float/path`），类型不匹配时会直接返回 404 错误，避免处理函数接收到无效参数：

- `/user/{user_id:int}`：仅匹配 `user_id` 为整数的路径（如 `/user/123`），`/user/abc` 会返回 404；
- `/user/{path:path}`：匹配任意子路径（如 `/user/a/b/c`），参数会接收完整的子路径字符串。

5.3 请求方法的严格匹配

每个路由规则都绑定了特定的请求方法（默认 GET），若客户端使用不支持的方法访问，会返回 405 (Method Not Allowed) 错误：

```
# 仅支持 POST 方法的路由
app.router.add_route('/api/post', lambda req: JSONResponse({'msg': 'POST 请求'}), methods=
['POST'])
```

- 用 GET 方法访问 `/api/post` → 返回 405 错误；
- 用 POST 方法访问 `/api/post` → 正常响应。

5.4 路由注册顺序影响匹配结果

当多个路由规则的路径模板存在重叠时，**先注册的路由会优先匹配**。因此，建议将**精确路由**和**高频访问路由**优先注册，避免被模糊路由覆盖：

```
# 先注册精确路由
@ui.page('/user/admin')
def user_admin():
    ui.label('管理员页面')

# 后注册模糊路由
@ui.page('/user/{user_id}')
def user_detail(user_id: str):
    ui.label(f'用户 {user_id}')
```

- 访问 `/user/admin` → 匹配 `user_admin`（先注册的精确路由），而非 `user_detail`。

六、app.router 的常见问题与解决方案

6.1 路由冲突导致页面无法访问

问题：注册的路由被其他路由覆盖，访问时跳转到错误的处理函数。

解决方案：

1. 遵循**精确路由优先注册**的原则，将精确路径的路由放在模糊路径之前；
2. 为路由添加唯一的 `name` 参数，避免名称冲突；
3. 通过 `print(app.router.routes)` 打印所有路由规则，检查是否存在重叠。

6.2 路径参数解析失败

问题：访问带参数的路由时返回 404，或处理函数未接收到参数。

解决方案：

1. 检查路径参数的类型注解是否与访问的路径匹配（如 `int` 类型参数不能传递字符串）；
2. 确保处理函数的参数名与路径模板中的参数名完全一致；
3. 避免在路径中使用特殊字符（如空格、中文），需进行 URL 编码。

6.3 自定义 API 接口无法生成文档

问题：使用 `app.router.add_api_route` 注册的接口未出现在 `/docs` 文档中。

解决方案：

1. 确保使用的是 `add_api_route` 而非 `add_route` (`add_route` 不支持 OpenAPI 文档)；
2. 为接口添加 `response_model`、`summary`、`description` 等参数，补充文档信息；
3. 检查 FastAPI 的版本是否与 NiceGUI 兼容，避免版本冲突。

6.4 WebSocket 路由连接失败

问题：客户端连接自定义 WebSocket 路由时返回 403/404 错误。

解决方案：

1. 检查 WebSocket 路由的路径是否正确，确保使用 `ws://` 或 `wss://` 协议；
2. 确保处理函数中调用了 `await websocket.accept()` 完成连接握手；
3. 若启用了跨域中间件，需为 WebSocket 配置跨域允许 (`CORSMiddleware` 已支持 WebSocket)。

6.5 404 异常处理函数不生效

问题：自定义的 404 异常处理函数未替换框架默认的错误页面。

解决方案：

1. 确保异常处理函数的参数包含 `request` 和 `exc`，且返回合法的响应对象；
2. 检查异常处理函数的注册顺序，需在路由注册前完成；
3. 避免在异常处理函数中抛出新的异常，导致框架回退到默认处理。

七、总结

`app.router` 是 NiceGUI 应用路由管理的核心中枢，底层基于 Starlette/FastAPI 的路由系统，实现了页面路由、API 路由、WebSocket 路由的统一管理。通过 `ui.page` 装饰器可快速注册前端页面路由，满足可视化界面开发的需求；通过 `app.router` 的原生 API 可手动注册自定义 API 接口、静态资源路由，实现前后端分离的全栈开发。同时，`app.router` 支持路由分组、依赖注入、异常处理等高级特性，可满足企业级应用的复杂路由需求。

开发中的最佳实践总结：

1. **路由分层：**页面路由使用 `ui.page`，API 路由使用 `app.router.add_api_route`，实现前后端逻辑分离；
2. **命名规范：**为路由添加唯一的 `name` 参数，通过 `url_path_for` 生成 URL，避免硬编码；
3. **匹配规则：**精确路由优先注册，避免模糊路由覆盖，参数添加类型约束；
4. **异常处理：**自定义 404/500 异常处理函数，提升用户体验；
5. **文档化：**为 API 路由添加响应模型和描述，利用 FastAPI 自动生成接口文档。

通过合理使用 `app.router`，可构建出结构清晰、扩展性强的 NiceGUI 应用，充分发挥框架在前端页面和后端 API 开发中的双重优势。

ui.page和app.router对比

要理清 NiceGUI 中 `ui.page` 和 `app.router` 的关系与差异，核心结论先明确：

`ui.page` 是快捷装饰器 / 函数，是开发者定义前端页面路由的「入口」；

`app.router` 是底层路由管理器，是 NiceGUI 管理所有页面路由的「核心引擎」。

`ui.page` 本质是对 `app.router` 核心能力的封装，二者是「上层接口」与「底层实现」的关系，但在使用方式、扩展能力上仍有明确区分。以下从核心定位、使用方式、能力边界、底层关联等维度展开详细对比。

一、核心定位与设计目标

维度	ui.page	app.router
核心角色	开发者友好的页面路由定义入口（上层装饰器 / 函数）	NiceGUI 内置的前端路由管理引擎（底层核心）
设计目标	简化页面路由注册流程，让开发者快速绑定 URL 到 UI 页面函数	管理所有页面路由的映射关系、处理前端导航请求、解析 URL 参数、控制页面跳转
面向用户	普通开发者（快速实现页面路由）	进阶开发者（自定义路由规则、扩展路由能力）
依赖关系	完全依赖 <code>app.router</code> 实现底层逻辑（ <code>ui.page</code> 调用 <code>app.router.add</code> ）	底层基于 Starlette 路由，独立管理前端页面路由映射

二、基本使用方式对比

1. 最简示例（直观区分）

`ui.page`：快速定义页面路由（99% 场景的首选）

`ui.page` 是装饰器 / 函数，直接绑定 URL 到页面函数，无需直接操作 `app.router`：

```
from nicegui import ui

# 方式1: 装饰器（最常用）
@ui.page('/') # 等价于调用 app.router.add('/', home_page)
def home_page():
    ui.label('首页')

# 方式2: 函数调用（适配动态注册场景）
def about_page():
    ui.label('关于页')
ui.page('/about')(about_page) # 手动调用装饰器，本质还是调用 app.router.add

ui.run()
```

`app.router`：底层管理路由（进阶场景）

`app.router` 是 `app` 实例的属性，可直接操作路由映射、控制跳转、解析参数：

```
from nicegui import app, ui

# 1. 直接通过 app.router 注册路由（等价于 ui.page）
def home_page():
    ui.label('首页')
app.router.add(path='/', endpoint=home_page) # ui.page 底层就是调用这行代码

# 2. 控制前端页面跳转（app.router 核心能力）
@ui.page('/')
def home():
    # 触发 app.router 的跳转逻辑
    ui.button('跳转到关于页', on_click=lambda: app.router.open('/about'))

# 3. 解析 URL 参数（app.router 解析后注入页面函数）
@ui.page('/user/{user_id}')
def user_page(user_id: str):
    # app.router 已解析路径参数，直接注入函数
    ui.label(f'用户ID: {user_id}')
    # 也可通过 app.router 读取当前路由信息
    ui.label(f'当前路径: {app.router.current_path}')

ui.run()
```

2. 路由注册方式对比

特性	<code>ui.page</code>	<code>app.router</code>
注册形式	装饰器（主流）/ 函数调用	方法调用（ <code>app.router.add()</code> ）
入参简化	仅需指定 <code>path</code> ，其他参数（如 <code>title</code> ）有默认值	<code>add()</code> 方法需显式传 <code>path / endpoint</code> ，支持更多底层参数
动态注册	支持（ <code>ui.page(path)(func)</code> ），但语法稍繁琐	原生支持动态注册（ <code>app.router.add(path, func)</code> ），更灵活
路由覆盖	重复注册同路径会静默覆盖（与 <code>app.router</code> 一致）	重复 <code>add()</code> 同路径会覆盖原有路由（无警告）
路由删除	无直接接口，需通过 <code>app.router</code> 操作	支持 <code>app.router.remove(path)</code> 删除指定路由

三、核心能力对比

1. 路由定义能力

特性	ui.page	app.router
路径参数	支持 (如 /user/{user_id}) , 但参数解析由 app.router 完成	核心负责解析路径参数, 注入页面函数; 可通过 app.router.resolve(path) 手动解析
页面标题	支持 title 参数 (如 @ui.page('/', title='首页'))	无直接设置标题的能力, 标题是 ui.page 封装的上层特性
路由优先级	无显式优先级, 按注册顺序匹配 (由 app.router 决定)	匹配规则由 app.router 控制 (如静态路径优先于动态路径)
路由列表	无直接获取方式, 需通过 app.router.routes 读取	可通过 app.router.routes 获取所有已注册的页面路由映射

2. 导航与跳转能力

特性	ui.page	app.router
页面跳转	无跳转能力, 仅定义页面	核心能力: open() (跳转)、back() (返回)、forward() (前进)、replace() (替换历史)
跳转参数传递	无传递能力, 需通过 app.router.open() 带参	支持 app.router.open('/path?key=value') 传递查询参数, 或 app.router.open('/path', params={'key': 'value'}) 传递内部参数
路由守卫	无原生支持, 需通过 app.router 扩展	可通过 app.router.before_navigate / after_navigate 钩子实现路由守卫 (如登录校验)
当前路由信息	无直接获取方式, 需通过 app.router 读取	支持 app.router.current_path (当前路径)、app.router.current_params (当前参数)

3. 扩展与自定义能力

特性	ui.page	app.router
自定义路由规则	无能力, 仅支持固定语法	可自定义路由匹配规则 (如正则路径)、扩展参数解析逻辑
路由钩子	无直接钩子, 需通过 app / app.router 绑定	支持 before_navigate / after_navigate 钩子 (页面跳转前后触发)
路由分组 / 前缀	无原生支持, 需手动拼接 URL	无原生分组能力, 但可通过自定义 add() 逻辑实现前缀统一

特性	ui.page	app.router
底层扩展	无，封装后屏蔽底层细节	可直接对接 Starlette 路由底层（如 app.router.routes 是 Starlette 路由列表）

四、底层关联与调用逻辑

ui.page 与 app.router 的核心关联可通过伪代码理解：

```
# ui.page 装饰器的底层实现逻辑（简化版）
def page(path: str, title: str = None):
    def decorator(func):
        # 1. 封装页面函数（如设置标题）
        def wrapped_func(*args, **kwargs):
            if title:
                ui.head_html(f'<title>{title}</title>')
            return func(*args, **kwargs)
        # 2. 调用 app.router.add 注册路由
        app.router.add(path=path, endpoint=wrapped_func)
        return wrapped_func
    return decorator
```

可见：

- ui.page 是 app.router.add 的「语法糖」，额外封装了标题、HTML 头信息等上层特性；
- 所有页面路由的注册、解析、跳转最终都由 app.router 完成；
- ui.page 屏蔽了 app.router 的底层细节，让普通开发者无需关注路由引擎的实现。

五、使用场景与最佳实践

1. ui.page 适用场景

- 99% 的普通场景：快速定义页面路由，绑定 URL 到 UI 页面函数；
- 需要设置页面标题、自定义 HTML 头信息（如 meta 标签）；
- 无需自定义路由规则，仅需基础的页面跳转和参数传递。

2. app.router 适用场景

- 动态注册 / 删除路由（如运行时根据配置添加页面）；
- 自定义页面跳转逻辑（如全局路由守卫、登录校验）；
- 手动解析路由参数、获取当前路由状态（如 current_path）；
- 扩展路由能力（如自定义参数解析规则、对接 Starlette 底层路由）。

3. 最佳实践示例（结合使用）

```
from nicegui import app, ui

# 1. 用 ui.page 快速定义页面（主流场景）
@ui.page('/', title='首页')
def home_page():
    ui.label('欢迎来到首页')
    # 用 app.router 控制跳转（核心能力）
    ui.button('跳转到用户页', on_click=lambda: app.router.open('/user/123?name=张三'))

# 2. 用 ui.page 定义带参数的页面，app.router 解析参数
@ui.page('/user/{user_id}')
def user_page(user_id: str):
    # 通过 app.router 获取当前路由信息
    current_path = app.router.current_path
    query_params = app.request.query_params # 获取查询参数
    ui.label(f'用户ID: {user_id}')
    ui.label(f'当前路径: {current_path}')
    ui.label(f'查询参数 name: {query_params.get("name")}')
    # 用 app.router 实现返回上一页
    ui.button('返回', on_click=app.router.back)

# 3. 进阶：用 app.router 动态注册路由（运行时添加）
def dynamic_page():
    ui.label('这是动态注册的页面')
# 运行时添加路由（等价于 ui.page('/dynamic')(dynamic_page)）
app.router.add(path='/dynamic', endpoint=dynamic_page)

# 4. 进阶：路由守卫（通过 app.router 钩子）
@app.router.before_navigate
async def before_navigate(path: str):
    # 校验：访问 /admin 需登录（示例）
    if path == '/admin' and not app.storage.user.get('logged_in'):
        app.router.open('/login') # 跳转到登录页
        return False # 阻止原跳转

ui.run()
```

六、总结

维度	ui.page	app.router
关系定位	上层快捷接口（装饰器 / 函数）	底层路由引擎（核心管理器）
核心作用	简化页面路由注册，封装标题等上层特性	管理路由映射、处理跳转、解析参数、控制导航
使用难度	低（开发者友好，无需关注底层）	中高（需了解路由底层逻辑，适合进阶场景）
扩展能力	弱（仅支持封装好的特性）	强（可自定义路由规则、扩展钩子、对接底层）
典型使用场景	快速定义页面、设置标题	动态注册路由、控制跳转、路由守卫、解析参数

简单来说：

- 日常开发中，**优先用 `ui.page` 定义页面路由**（简单、高效）；
- 当需要**控制页面跳转、解析参数、自定义路由规则**时，直接操作 `app.router`；
- `ui.page` 是 `app.router` 的「便捷包装」，而 `app.router` 是所有前端页面路由的「最终执行者」。

NiceGUI APIRouter 的 page 方法深度解析

在 NiceGUI 中，`APIRouter.page()` 是 `APIRouter` 扩展的核心方法之一，它打破了「`APIRouter` 仅管理 API、`ui.page` 仅管理页面」的单一边界，允许将**前端页面路由**也纳入 `APIRouter` 的模块化管理体系中，实现「API 路由 + 页面路由」的一体化模块化拆分。本文从方法定义、核心特性、使用场景到高级配置，全面解析该方法的使用。

一、page 方法的核心定位

1. 方法本质

`APIRouter.page()` 是 `ui.page()` 的「模块化封装版」，作用是：

- 为指定的 `APIRouter` 实例注册**前端页面路由**；
- 页面路由会继承 `APIRouter` 的路径前缀，实现页面路由的模块化隔离；
- 最终与 `APIRouter` 的 API 路由一起，通过 `app.include_router()` 批量挂载到主应用。

2. 与 ui.page () 的核心区别

维度	<code>APIRouter.page()</code>	<code>ui.page()</code>
归属	隶属于某个 <code>APIRouter</code> 实例，模块化管理	全局注册，直接归属主应用
路径前缀	自动叠加 <code>APIRouter</code> 的 <code>prefix</code>	无默认前缀，需手动指定
挂载方式	随 <code>APIRouter</code> 批量挂载（ <code>app.include_router()</code> ）	自动注册到主应用，无需显式挂载
使用场景	模块化页面（如用户模块页面、订单模块页面）	全局通用页面（如首页、404 页面）

3. 方法签名（简化版）

```
def page(  
    self,  
    path: str, # 页面路径（会叠加 APIRouter 的 prefix）  
    *,  
    title: str | None = None, # 页面标题  
    icon: str | None = None, # 页面图标（Font Awesome 图标名）  
    dark: bool | None = None, # 是否启用暗黑模式  
    response_timeout: float = 3.0, # 响应超时时间  
)-> Callable[[Callable], Callable]:  
    ...
```

参数与 `ui.page()` 基本一致，核心差异是「路径会叠加 `APIRouter` 的前缀」。

二、基础使用步骤

1. 最简示例：模块化页面路由

步骤 1：创建 APIRouter 并注册页面

```
# main.py
from nicegui import APIRouter, app, ui

# 1. 创建 APIRouter 实例，指定前缀 /user
user_router = APIRouter(prefix="/user")

# 2. 用 APIRouter.page() 注册页面（路径叠加前缀后为 /user/profile）
@user_router.page("/profile")
def user_profile_page():
    ui.label("用户个人中心").classes("text-2xl")
    ui.input("用户名", value="张三").classes("mt-4")
    ui.button("保存").classes("mt-2")

# 3. 同时注册该模块的 API 路由（可选，体现一体化）
@user_router.get("/api/info")
def get_user_info():
    return {"name": "张三", "age": 25}

# 4. 挂载 APIRouter 到主应用
app.include_router(user_router)

# 全局首页（ui.page() 注册）
@ui.page("/")
def home_page():
    ui.label("首页").classes("text-3xl")
    ui.link("进入个人中心", "/user/profile").classes("mt-4")

if __name__ in {"__main__", "__mp_main__"}:
    ui.run()
```

步骤 2：访问验证

- 页面路径：http://localhost:8080/user/profile（叠加了 APIRouter 的 /user 前缀）；
- API 路径：http://localhost:8080/user/api/info（同样叠加前缀）；
- 首页路径：http://localhost:8080/（全局页面，无前缀）。

三、核心特性

1. 路径前缀自动叠加

这是 `APIRouter.page()` 最核心的特性，示例：

```
# 定义 APIRouter 时指定前缀 /api/v1/order
order_router = APIRouter(prefix="/api/v1/order")

# 注册页面路径 /detail → 最终路径 /api/v1/order/detail
@order_router.page("/detail")
def order_detail_page():
    ui.label("订单详情页")

# 若 APIRouter 挂载时额外指定前缀，会二次叠加：
app.include_router(order_router, prefix="/admin")
# 最终页面路径： /admin/api/v1/order/detail
```

2. 一体化管理 API + 页面

一个 `APIRouter` 可同时管理该模块的 API 路由和页面路由，实现「业务模块闭环」：

```
# 订单模块路由：API + 页面一体化
order_router = APIRouter(prefix="/order")

# 订单模块 API
@order_router.get("/api/list")
def get_order_list():
    return [{"id": 1, "amount": 99.9}, {"id": 2, "amount": 199.9}]

# 订单模块页面（展示订单列表）
@order_router.page("/list")
def order_list_page():
    ui.label("订单列表").classes("text-2x1")
    # 调用本模块的 API 获取数据
    order_data = ui.run_javascript("fetch('/order/api/list').then(r => r.json())")
    ui.table(
        columns=["订单ID", "金额"],
        rows=order_data,
    ).classes("mt-4")

# 挂载后：
# API 路径： /order/api/list
# 页面路径： /order/list
```

3. 支持所有 `ui.page()` 的特性

`APIRouter.page()` 完全继承 `ui.page()` 的所有功能，包括：

- 路径参数（如 `/order/{order_id}`）；
- 页面标题、图标、暗黑模式；
- 异步页面函数（`async def`）；
- 页面级的响应超时配置。

示例（带路径参数的页面）：

```

order_router = APIRouter(prefix="/order")

# 页面路径: /order/detail/{order_id}
@order_router.page("/detail/{order_id}")
def order_detail_page(order_id: str):
    ui.label(f"订单详情 - {order_id}").classes("text-2xl")
    # 调用 API 获取该订单详情
    order_info = ui.run_javascript(f"fetch('/order/api/detail/{order_id}').then(r =>
r.json())")
    ui.json(order_info).classes("mt-4")

# 对应 API
@order_router.get("/api/detail/{order_id}")
def get_order_detail(order_id: str):
    return {"order_id": order_id, "status": "paid", "amount": 99.9}

```

4. 与 API 路由共享中间件

为 `APIRouter` 添加的中间件，会同时作用于该路由下的**所有 API 路由和页面路由**，实现模块化的权限控制、日志记录等。

示例（模块级权限校验）：

```

from nicegui import APIRouter, Request, Response

# 订单模块路由
order_router = APIRouter(prefix="/order")

# 定义中间件：校验登录状态
async def auth_middleware(request: Request, call_next):
    # 从 Cookie/Header 获取登录态
    token = request.cookies.get("token") or request.headers.get("Authorization")
    if not token:
        # 未登录则重定向到登录页
        return Response(status_code=307, headers={"Location": "/login"})
    response = await call_next(request)
    return response

# 为 APIRouter 添加中间件（作用于所有 API 和页面）
order_router.add_middleware(auth_middleware)

# 受保护的页面
@order_router.page("/list")
def order_list_page():
    ui.label("订单列表（需登录）")

# 受保护的 API
@order_router.get("/api/list")
def get_order_list():
    return [{"id": 1}]

# 全局登录页
@ui.page("/login")

```

```
def login_page():
    ui.label("请登录").classes("text-2x1")
    ui.input("Token").bind_value_to(ui.state, "token")
    ui.button("登录", on_click=lambda: ui.cookies.set("token", ui.state.token))
```

此时访问 `/order/list` 或 `/order/api/list`，未登录会自动重定向到 `/login`。

四、高级使用场景

1. 多模块页面拆分（大型项目）

将不同业务模块的页面和 API 统一放到对应的 `APIRouter` 中，实现极致的模块化：

```
project/
├─ main.py                # 入口：挂载所有路由
├─ modules/
│   └─ user/
│       └─ router.py      # 用户模块 APIRouter（含 API + 页面）
│       └─ components.py  # 用户模块专属组件
│   └─ order/
│       └─ router.py      # 订单模块 APIRouter
│       └─ components.py  # 订单模块专属组件
└─ static/                # 静态资源
```

(1) 用户模块路由（modules/user/router.py）

```
from nicegui import APIRouter
from .components import user_info_card

# 用户模块路由：前缀 /user
user_router = APIRouter(prefix="/user")

# 用户模块页面
@user_router.page("/profile")
def user_profile_page():
    user_info_card() # 复用模块内组件

# 用户模块 API
@user_router.get("/api/info")
def get_user_info():
    return {"name": "张三", "age": 25}
```

(2) 订单模块路由（modules/order/router.py）

```
from nicegui import APIRouter
from .components import order_table

# 订单模块路由：前缀 /order
order_router = APIRouter(prefix="/order")

# 订单模块页面
```

```
@order_router.page("/list")
def order_list_page():
    order_table() # 复用模块内组件

# 订单模块 API
@order_router.get("/api/list")
def get_order_list():
    return [{"id": 1, "amount": 99.9}]
```

(3) 主应用挂载 (main.py)

```
from nicegui import app, ui
from modules.user.router import user_router
from modules.order.router import order_router

# 挂载所有模块路由 (API + 页面)
app.include_router(user_router)
app.include_router(order_router)

# 全局首页
@ui.page("/")
def home_page():
    ui.label("首页").classes("text-3xl")
    ui.link("用户中心", "/user/profile").classes("mr-4")
    ui.link("订单列表", "/order/list")

if __name__ in {"__main__", "__mp_main__"}:
    ui.run()
```

2. 页面路由的动态注册

`APIRouter.page()` 支持动态注册页面 (如根据配置生成页面路由), 灵活性远超全局 `ui.page()`:

```
from nicegui import APIRouter, app, ui

# 动态生成页面配置
page_configs = [
    {"path": "/dashboard", "title": "数据看板"},
    {"path": "/settings", "title": "系统设置"},
]

# 创建管理模块路由
admin_router = APIRouter(prefix="/admin")

# 动态注册页面
for config in page_configs:
    @admin_router.page(config["path"], title=config["title"])
    def dynamic_page(config=config): # 闭包需捕获 config
        ui.label(config["title"]).classes("text-2xl")

app.include_router(admin_router)
```

```
# 访问 /admin/dashboard → 数据看板页面
# 访问 /admin/settings → 系统设置页面
```

3. 页面与 API 的联动优化

通过 `APIRouter.page()` 注册的页面，可通过 `app.request_client` 直接调用同模块的 API（服务端调用，避免跨域 / 网络开销）：

```
from nicegui import APIRouter, app, ui

order_router = APIRouter(prefix="/order")

# 订单 API
@order_router.get("/api/detail/{order_id}")
def get_order_detail(order_id: str):
    return {"order_id": order_id, "amount": 99.9}

# 订单页面（服务端调用 API）
@order_router.page("/detail/{order_id}")
async def order_detail_page(order_id: str):
    # 服务端调用同模块 API，无需走网络
    response = await app.request_client.get(f"/order/api/detail/{order_id}")
    order_data = await response.json()

    ui.label(f"订单 {order_id} 详情").classes("text-2xl")
    ui.json(order_data).classes("mt-4")
```

五、注意事项与最佳实践

1. 路径冲突规避

- `APIRouter.page()` 注册的页面路径会叠加前缀，需避免与其他 `APIRouter` 或全局 `ui.page()` 的路径冲突；
- 建议为不同模块的 `APIRouter` 设置唯一前缀（如 `/user`、`/order`、`/admin`）。

2. 中间件作用域

- 为 `APIRouter` 添加的中间件会作用于该路由下的**所有页面和 API**，若需仅作用于 API，可拆分两个 `APIRouter`（一个管 API，一个管页面）；
- 全局中间件（`app.add_middleware()`）会作用于所有路由（包括 `APIRouter.page()` 注册的页面）。

3. 异步优先

`APIRouter.page()` 支持异步页面函数（`async def`），建议与异步 API 函数配合使用，避免阻塞事件循环：

```
@order_router.page("/list")
async def order_list_page():
    # 异步调用 API
    response = await app.request_client.get("/order/api/list")
    order_data = await response.json()
    ui.table(columns=["ID", "金额"], rows=order_data)
```

4. 页面资源隔离

模块化页面建议使用 `ui.card()`、`ui.column()` 等容器组件包裹，避免样式 / 组件冲突；复杂组件可抽离到模块内的 `components.py` 中。

5. 生产环境部署

- `APIRouter.page()` 注册的页面与 `ui.page()` 页面的部署方式完全一致；
- 若使用反向代理（如 Nginx），需确保路径转发包含 `APIRouter` 的前缀（如 `/user/`、`/order/`）。

六、常见误区

误区 1：认为 `page()` 是 API 方法

`APIRouter.page()` 仅用于注册前端页面，而非 API，若在该方法装饰的函数中返回 JSON，返回值会被忽略（页面仍渲染 HTML）：

```
# 错误示例
@order_router.page("/api/wrong")
def wrong_page():
    return {"error": "wrong"} # 返回值无效，页面显示空 HTML
```

误区 2：忽略前缀叠加导致路径错误

```
# 定义时
user_router = APIRouter(prefix="/user")
@user_router.page("/user/profile") # 错误：叠加后路径为 /user/user/profile
def user_profile_page():
    ...

# 正确写法
@user_router.page("/profile") # 叠加后路径为 /user/profile
def user_profile_page():
    ...
```

误区 3：中间件未处理页面重定向

若中间件返回 `Response`（如重定向），需确保状态码和 Header 正确（如 307 临时重定向），否则页面可能无法正常跳转：

```
# 正确的重定向中间件
async def auth_middleware(request: Request, call_next):
    if not request.cookies.get("token"):
        # 307 保留请求方法，302 可能丢失 POST 等方法（页面路由多为 GET，影响较小）
        return Response(status_code=307, headers={"Location": "/login"})
    return await call_next(request)
```

七、总结

`APIRouter.page()` 是 NiceGUI 模块化开发的「进阶利器」，核心价值在于：

- 1. **一体化管理**：将模块内的「API 路由 + 页面路由」统一纳入 `APIRouter`，实现业务模块的闭环；
- 2. **前缀隔离**：自动叠加 `APIRouter` 前缀，避免页面路径冲突；
- 3. **中间件复用**：模块级中间件同时作用于页面和 API，简化权限、日志等逻辑的统一管控；
- 4. **模块化拆分**：支持大型项目按业务模块拆分页面和 API，提升代码可维护性。

掌握该方法后，你可以告别「全局页面 + 分散 API」的混乱模式，构建出结构清晰、职责明确的 NiceGUI 应用，尤其适合中大型项目的开发与维护。

ui.page 与 APIRouter.page 深度对比

`ui.page` 和 `APIRouter.page` 是 NiceGUI 中注册前端页面路由的两种核心方式，二者底层同源（均基于 Starlette 路由），但定位、使用场景和特性差异显著。本文从**核心定位**、**语法特性**、**模块化能力**、**工程化适配**等维度，全方位对比二者的区别与适用场景，帮助你精准选择页面注册方式。

一、核心维度对比总表

对比维度	ui.page（全局页面注册）	APIRouter.page（模块化页面注册）
核心定位	全局页面路由注册，无模块归属	模块化页面路由注册，隶属于指定 APIRouter 实例
路径规则	路径为「绝对路径」，无自动前缀	路径为「相对路径」，自动叠加 APIRouter 的 prefix
挂载方式	装饰器自动注册到主应用，无需显式挂载	随所属 APIRouter 一起通过 <code>app.include_router()</code> 挂载
模块化能力	弱（全局注册，无模块隔离）	强（与 API 路由同模块，支持业务闭环）
中间件支持	仅支持全局中间件（ <code>app.add_middleware</code> ）	支持「模块级中间件 + 全局中间件」
复用性	仅全局复用，无法按模块拆分	模块内复用，可与同模块 API / 组件联动
路径冲突风险	高（所有页面共用全局路径空间）	低（前缀隔离，模块内路径独立）
工程化适配	适合小型项目 / 全局通用页面	适合中大型项目 / 业务模块内页面

对比维度	ui.page（全局页面注册）	APIRouter.page（模块化页面注册）
参数继承	仅继承全局配置（如 dark/title 需手动指定）	可继承 APIRouter 前缀，模块内参数统一管控
动态注册灵活性	弱（全局注册，动态添加易混乱）	强（模块内动态注册，不影响全局）

二、关键特性深度对比

1. 路径规则（最核心差异）

路径处理是二者最本质的区别，直接决定了页面路由的组织方式：

(1) ui.page：绝对路径，无前缀叠加

ui.page 的路径是「全局绝对路径」，写什么路径就是最终访问路径，无自动前缀：

```
# 最终访问路径: /home
@ui.page("/home")
def home_page():
    ui.label("全局首页")

# 最终访问路径: /user/profile（需手动加前缀，易冲突）
@ui.page("/user/profile")
def user_profile_page():
    ui.label("用户中心")
```

(2) APIRouter.page：相对路径 + 前缀自动叠加

APIRouter.page 的路径是「相对于 APIRouter 前缀的相对路径」，最终路径 = APIRouter.prefix + 页面路径：

```
# 定义模块级 APIRouter，指定前缀 /user
user_router = APIRouter(prefix="/user")

# 页面相对路径 /profile → 最终路径 /user/profile
@user_router.page("/profile")
def user_profile_page():
    ui.label("用户中心")

# 若挂载时额外加前缀，会二次叠加：
app.include_router(user_router, prefix="/admin")
# 最终路径: /admin/user/profile
```

优势对比：

- ui.page：适合无模块隔离的简单页面，但手动加前缀易出错（如多个模块都写 /list 会冲突）；
- APIRouter.page：前缀自动隔离，模块内只需关注「相对路径」，无需担心全局冲突（如用户模块 /list、订单模块 /list 最终路径为 /user/list、/order/list）。

2. 挂载与注册机制

(1) ui.page：自动注册，无显式挂载

`ui.page` 装饰器执行时，会直接将页面路由注册到 NiceGUI 主应用的路由列表中，无需额外操作：

```
# 装饰器执行即完成注册，运行时已存在于 app.routes 中
@ui.page("/about")
def about_page():
    ui.label("关于页面")

# 无需手动挂载，直接访问 /about 即可
ui.run()
```

(2) APIRouter.page：需随 APIRouter 显式挂载

`APIRouter.page` 仅将页面路由注册到所属的 `APIRouter` 实例中，需通过 `app.include_router()` 挂载到主应用才生效：

```
# 步骤1: 创建 APIRouter 并注册页面（仅存于 router 实例，未生效）
order_router = APIRouter(prefix="/order")
@order_router.page("/list")
def order_list_page():
    ui.label("订单列表")

# 步骤2: 显式挂载后，页面路由才生效
app.include_router(order_router)

ui.run()
```

优势对比：

- `ui.page`：简单快捷，适合小型项目；
- `APIRouter.page`：支持「延迟挂载」「条件挂载」（如根据配置决定是否挂载某个模块），灵活性更高。

3. 中间件支持（权限 / 日志等关键能力）

中间件是实现权限校验、日志记录、请求拦截的核心，二者的支持范围差异显著：

(1) ui.page：仅支持全局中间件

`ui.page` 注册的页面只能受「全局中间件」管控，无法为特定页面单独配置中间件：

```
# 全局中间件：作用于所有页面（包括 ui.page 和 APIRouter.page）
async def global_middleware(request: Request, call_next):
    print(f"全局拦截: {request.url}")
    return await call_next(request)
app.add_middleware(global_middleware)

# ui.page 页面无法单独配置中间件
@ui.page("/admin")
def admin_page():
    ui.label("管理员页面") # 只能受全局中间件管控
```

(2) APIRouter.page：支持模块级 + 全局中间件

`APIRouter.page` 注册的页面，既受全局中间件管控，也受所属 `APIRouter` 的「模块级中间件」管控：

```
# 步骤1：定义模块级中间件（仅作用于该 APIRouter 的页面/API）
async def auth_middleware(request: Request, call_next):
    if not request.cookies.get("token"):
        return Response(status_code=307, headers={"Location": "/login"})
    return await call_next(request)

# 步骤2：为 APIRouter 添加模块级中间件
admin_router = APIRouter(prefix="/admin")
admin_router.add_middleware(auth_middleware)

# 步骤3：注册页面（自动继承模块级中间件）
@admin_router.page("/dashboard")
def admin_dashboard():
    ui.label("管理员看板") # 需登录才能访问

# 挂载后，该页面同时受「全局中间件 + 模块级 auth_middleware」管控
app.include_router(admin_router)
```

优势对比：

- `ui.page`：无模块级中间件能力，权限控制需在页面内硬编码（如 `if not token: ui.navigate.to("/login")`），代码冗余；
- `APIRouter.page`：模块级中间件可统一管控该模块的所有页面 / API，权限逻辑集中维护，代码更简洁。

4. 模块化与工程化适配

(1) ui.page：适合小型项目，工程化能力弱

`ui.page` 是「全局注册模式」，当项目规模扩大时，会出现以下问题：

- 所有页面路由集中在全局，代码分散（如用户页面、订单页面混在一起）；
- 页面与对应 API 路由无关联，需手动维护路径对应关系；
- 路径冲突风险高（如多个模块都定义 `/detail` 页面）。

示例（混乱的全局注册）：

```
# main.py 中混杂所有页面，代码臃肿
@ui.page("/user/profile") # 手动加 /user 前缀
def user_profile(): ...

@ui.page("/order/detail") # 手动加 /order 前缀
def order_detail(): ...

@ui.page("/admin/dashboard") # 手动加 /admin 前缀
def admin_dashboard(): ...
```

(2) APIRouter.page: 适合中大型项目，工程化能力强

APIRouter.page 支持「按业务模块拆分页面 + API」，实现代码的模块化隔离：

```
project/
├─ main.py           # 仅挂载路由
├─ modules/
│   ├─ user/
│   │   ├─ router.py    # user_router (含页面 + API)
│   │   └─ components.py # 用户模块组件
│   ├─ order/
│   │   ├─ router.py    # order_router (含页面 + API)
│   │   └─ components.py # 订单模块组件
│   └─ admin/
│       ├─ router.py    # admin_router (含页面 + API)
│       └─ components.py # 管理员模块组件
```

示例（模块化注册）：

```
# modules/user/router.py
user_router = APIRouter(prefix="/user")
@user_router.page("/profile") # 模块内相对路径
def user_profile(): ... # 复用同模块组件
@user_router.get("/api/info") # 同模块 API
def get_user_info(): ...

# modules/order/router.py
order_router = APIRouter(prefix="/order")
@order_router.page("/detail") # 模块内相对路径
def order_detail(): ... # 复用同模块组件
@order_router.get("/api/detail") # 同模块 API
def get_order_detail(): ...

# main.py
app.include_router(user_router)
app.include_router(order_router)
```

优势对比：

- `ui.page`：无模块化隔离，代码易混乱，仅适合单文件小型应用；
- `APIRouter.page`：模块内页面 / API / 组件闭环，代码结构清晰，适配中大型项目的团队协作与维护。

5. 动态注册与灵活性

(1) ui.page：动态注册易污染全局

`ui.page` 支持动态注册，但会直接修改全局路由列表，易导致冲突：

```
# 动态注册页面（全局路径，易冲突）
def dynamic_register():
    @ui.page("/dynamic-page")
    def dynamic_page():
        ui.label("动态页面")

dynamic_register() # 注册后全局可见，若重复调用会冲突
```

(2) APIRouter.page：动态注册仅影响模块

`APIRouter.page` 动态注册的页面仅属于当前模块，不会污染全局：

```
# 模块内动态注册页面
admin_router = APIRouter(prefix="/admin")

page_configs = ["/dashboard", "/settings", "/logs"]
for path in page_configs:
    @admin_router.page(path)
    def dynamic_page(path=path):
        ui.label(f"管理员页面: {path}")

app.include_router(admin_router) # 仅挂载该模块，不影响全局
```

三、适用场景与最佳实践

1. ui.page 适用场景

- **小型项目 / 单文件应用**：无需模块化拆分，快速实现页面；
- **全局通用页面**：如首页、登录页、404 页面、关于页等；
- **临时测试页面**：快速验证功能，无需考虑模块化。

2. APIRouter.page 适用场景

- **中大型项目**：按业务模块（用户、订单、商品）拆分页面；
- **需权限管控的模块页面**：如管理员页面、会员中心（模块级中间件统一鉴权）；
- **与 API 联动的页面**：页面与同模块 API 协同（如订单页面调用订单 API）；
- **团队协作开发**：不同开发者负责不同模块，避免代码冲突。

3. 最佳实践：二者结合使用

在实际项目中，建议「全局页面用 `ui.page`，模块页面用 `APIRouter.page`」：

```
from nicegui import ui, APIRouter, app

# 1. 全局通用页面 (ui.page)
@ui.page("/")
def home_page():
    ui.label("全局首页").classes("text-3xl")
    ui.link("用户中心", "/user/profile")
    ui.link("订单列表", "/order/list")

@ui.page("/login")
def login_page():
    ui.input("Token").bind_value_to(ui.state, "token")
    ui.button("登录", on_click=lambda: ui.cookies.set("token", ui.state.token))

# 2. 用户模块页面 + API (APIRouter.page)
user_router = APIRouter(prefix="/user")
@user_router.page("/profile")
def user_profile():
    ui.label("用户中心").classes("text-2xl")
@user_router.get("/api/info")
def get_user_info():
    return {"name": "张三"}

# 3. 订单模块页面 + API (APIRouter.page)
order_router = APIRouter(prefix="/order")
@order_router.page("/list")
def order_list():
    ui.label("订单列表").classes("text-2xl")
@order_router.get("/api/list")
def get_order_list():
    return [{"id": 1, "amount": 99.9}]

# 4. 挂载模块路由
app.include_router(user_router)
app.include_router(order_router)

ui.run()
```

四、常见误区与避坑指南

误区 1：用 `APIRouter.page` 注册全局页面

`APIRouter.page` 注册的页面会叠加前缀，不适合全局通用页面（如登录页）：

```
# 错误：登录页路径变为 /user/login（叠加了 /user 前缀）
user_router = APIRouter(prefix="/user")
@user_router.page("/login")
def login_page(): ...

# 正确：全局登录页用 ui.page
@ui.page("/login")
def login_page(): ...
```

误区 2：忽略 APIRouter.page 的前缀叠加导致路径错误

```
# 错误：叠加后路径为 /user/user/profile
user_router = APIRouter(prefix="/user")
@user_router.page("/user/profile")
def user_profile(): ...

# 正确：模块内用相对路径，最终路径 /user/profile
@user_router.page("/profile")
def user_profile(): ...
```

误区 3：为 ui.page 页面硬编码权限逻辑

ui.page 无模块级中间件，若需权限控制，建议改用 APIRouter.page：

```
# 不推荐：ui.page 内硬编码权限（代码冗余）
@ui.page("/admin")
def admin_page():
    if not ui.cookies.get("token"):
        ui.navigate.to("/login")
    ui.label("管理员页面")

# 推荐：APIRouter.page + 模块级中间件（逻辑集中）
admin_router = APIRouter(prefix="/admin")
admin_router.add_middleware(auth_middleware)
@admin_router.page("/")
def admin_page():
    ui.label("管理员页面")
```

五、总结

选择建议	优先用 ui.page	优先用 APIRouter.page
项目规模	小型 / 单文件	中大型 / 多模块
页面类型	全局通用页面（首页 / 登录页）	业务模块页面（用户 / 订单 / 商品）
权限管控	无需权限 / 简单权限	模块级统一权限管控
路径复杂度	路径少、无冲突	路径多、需前缀隔离
工程化需求	无团队协作 / 快速开发	团队协作 / 模块化维护

二者的核心差异本质是「全局化 vs 模块化」：

- `ui.page` 是「全局视角」，追求简单快捷，适合小场景；
- `APIRouter.page` 是「模块视角」，追求结构清晰、可维护性，适合复杂场景。

在实际开发中，结合二者的优势（全局页面用 `ui.page`，模块页面用 `APIRouter.page`），能最大化 NiceGUI 的开发效率和工程化能力。